

Conditional Aggregation Consolidation

#sql-server #performance #set-based #refactoring

What the Pattern Looks Like

A stored procedure runs several `Count(*)` (or `Sum`, `Max`, etc.) queries back to back at the top of the proc, each landing its result in a local variable. The queries either hit the same table with different filters, or hit different tables that share one or more parameter values. Each query is its own statement, with its own compile, its own plan, and its own round trip through the engine.

```
Declare @CountA Int = 0, @CountB Int = 0, @CountC Int = 0

Select @CountA = Count(*) From TableX With (NoLock)
Where AddrBank = @AddrBank And FillType = 'A' And OrderStatus = 'Counted'

Select @CountB = Count(*) From TableY With (NoLock)
Where AddrBank = @AddrBank And Status = 'Online'

Select @CountC = Count(*) From TableZ With (NoLock) Inner Join TableY ...
Where AddrBank = @AddrBank And RemainingTries > 0
```

Three separate statements, three plans, three result-set round trips, three sets of statistics lookups. The proc body reads as a sequence of independent counts, but the optimizer treats them as three independent queries with no shared work.

Why It Hurts (and How Much)

The cost depends on whether the queries hit overlapping data.

- **When the queries hit the same table with different predicates**, the engine reads the table multiple times. Each statement compiles its own plan, scans or seeks the table independently, and produces one count. The shared work (the table read itself) is done N times.
- **When the queries hit different tables but share parameters**, the engine still has to compile and dispatch each statement separately. The per-statement overhead (parse, compile, plan lookup, parameter binding, result return) is paid N times even though the work is logically one composite count.

In Query Store, the symptom is several distinct `query_ids` that all originate inside the same proc with similar `query_text` shapes. The total reads sum across them, but each individual statement may stay below the top-50 threshold and be invisible at the per-query level.

The other invisible cost is plan-cache pressure. Three statements occupy three plan slots in the cache, each with their own statistics dependencies and their own recompile triggers. One consolidated statement occupies one slot.

The Refactoring Steps

The fix has two flavors depending on whether the source data overlaps.

Flavor A: Same-table, different filters

When all the counts read the same table, conditional aggregation collapses them into a single scan:

```
Select
    @CountReady    = Sum(Case When Status = 'Ready' Then 1 Else 0 End),
    @CountWaiting  = Sum(Case When Status = 'Waiting' Then 1 Else 0 End),
```

```

@CountDone = Sum(Case When Status = 'Done' Then 1 Else 0 End)
From TableX With (NoLock)
Where AddrBank = @AddrBank

```

One scan, one statement, three counts. The shared Where AddrBank = @AddrBank predicate is evaluated once. The branch-specific predicates move into Case expressions inside the aggregate. The optimizer reads the table the minimum number of times required, which is once.

This is the highest-leverage form of the pattern. The base table scan is the dominant cost, and consolidation cuts it from N reads to one read.

Flavor B: Different tables, shared parameters

When the counts hit different tables, you cannot reduce the underlying I/O by consolidation alone. What you can reduce is the per-statement overhead. A UNION All of projection queries followed by conditional aggregation collapses the dispatch into one statement:

```

Select
@CountA = Sum(Case When Bucket = 'A' Then 1 Else 0 End),
@CountB = Sum(Case When Bucket = 'B' Then 1 Else 0 End),
@CountC = Sum(Case When Bucket = 'C' Then 1 Else 0 End)
From
(
  Select 'A' As Bucket From TableX With (NoLock)
  Where AddrBank = @AddrBank And FillType = 'A' And OrderStatus = 'Counted'
  Union All
  Select 'B' As Bucket From TableY With (NoLock)
  Where AddrBank = @AddrBank And Status = 'Online'
  Union All
  Select 'C' As Bucket From TableZ With (NoLock) Inner Join TableY On ...
  Where AddrBank = @AddrBank And RemainingTries > 0
) AS Sources

```

The work done at the leaf level is the same as the three-statement form. What changes:

- One compile instead of three.
- One plan-cache slot instead of three.
- One statement-completion round trip instead of three.
- The optimizer sees all three sources together and can sometimes find sharing opportunities (for example, when two of the sources hit the same index, the engine may consolidate the scans). This is opportunistic, not guaranteed.

Flavor B is a structural and cache-pressure win. It is not a leaf-I/O win. Be honest about which flavor you are getting when you write the analysis. A claim of “fewer reads” is only true for Flavor A.

When It Is Worth Doing

- The proc runs many times per second, where per-statement overhead matters.
- The counts feed into a single decision (an If block, a flag assignment, a switch statement) and there is no behavioral reason to keep them as separate statements.
- Two or more of the sources share a non-trivial filter that can be lifted into the outer query (Flavor A).

When It Is Not Worth Doing

- The counts feed into separate code paths that have their own early-exit logic. Consolidating forces the engine to compute counts that the proc would otherwise short-circuit on. If the proc returns early after

@CountA = 0 and never reads @CountB, the consolidated form pays for @CountB that the original skipped.

- The sources have wildly different data shapes (one table is 50 rows, another is 50 million). Forcing them through a single plan can produce a join order tuned to the average, which is bad for both. Three independent plans, each tuned to its own data, can outperform.
- The original counts use different NoLock semantics (rare, but possible). Consolidation requires consistent locking hints across the union.

Same-Pattern Variants

The technique is not limited to Count (*). The same shape works for:

- Sum(Amount) instead of Count (*), when summing different conditional buckets.
- Max(SomeDate) and Min(SomeDate) together, with Case expressions filtering each.
- Boolean flags via Max(Case When ... Then 1 Else 0 End), which is a common alternative to If Exists when you need several existence checks at once.

The boolean-flag variant is the natural pair for the related single-pass Exists rewrite: when a procedure asks “is X available?” and “is Y available?” against the same table, one pass with two conditional Maxes returns both flags.

Recognizing the Pattern

Open any proc and read the first twenty lines. If you see three or more Select @Var = ... assignments in a row, each with its own From clause, ask whether they share a Where predicate or a From table. If they do, the pattern applies. If they do not, but the proc runs at high frequency, Flavor B may still be worth the structural cleanup.

In code review, the smell is several adjacent variable-assignment counts. The fix is one statement.

Related Concepts

- **FOR XML PATH Consolidation**: the same materialize-once principle for the case where the repeated work is a correlated string-aggregation subquery rather than independent counts.
- **Correlated Subqueries to CTEs**: the broader pattern of replacing per-row evaluation with set-based evaluation.
- **UNION ALL Views**: when consolidating via Union All, the same labeling discipline applies. Use Union All (never Union) when the source-tag column makes the rows distinguishable.
- **Table Variables vs Temp Tables**: a related “do the work once” pattern for cases where the consolidated result is too large to land in a scalar variable.